

Integrated Development Environments (IDEs) Used

Component	IDE Used
Arduino Nano Controller	Arduino IDE
Arduino Uno Robot Cars	Arduino IDE
ESP32 Arena Controller	Arduino IDE
ESP32 Web Interface	VSCode (HTML, CSS, JavaScript embedded in ESP32 code)

Programming Languages Used

Component	Language
Arduino Nano Controller	C/C++ (Arduino Framework)
Arduino Uno Robot Cars	C/C++ (Arduino Framework)
ESP32 Arena Controller	C/C++ (Arduino Framework)
Arena Web Interface	HTML, CSS, JavaScript

Car Code

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>
#include <Servo.h>

RF24 radio(9, 10); // CE, CSN
const byte address[6] = "00001"; // change to 00002 for Car 2

Servo myServo;

// Data from transmitter
struct DataPacket {
    int x;
    int y;
    bool button;
```

```

    int x2;          // ignored by car
    bool goalieBtn; // ignored by car
};

DataPacket data;

// Motor driver pins (L298N)
const int ENA = 5;
const int IN1 = 2;
const int IN2 = 4;
const int IN3 = 7;
const int IN4 = 8;
const int ENB = 6;

// Servo
const int servoPin = 3;
bool prevButton = false;

void setup() {
    Serial.begin(9600);

    radio.begin();
    radio.openReadingPipe(0, address);
    radio.setPALevel(RF24_PA_LOW);
    radio.startListening();

    pinMode(ENA, OUTPUT);
    pinMode(ENB, OUTPUT);
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);

    myServo.attach(servoPin);
    myServo.write(0);

    Serial.println("Car Receiver Ready");
}

void loop() {
    if (radio.available()) {
        radio.read(&data, sizeof(data));
        handleMovement();
        handleServo();
    }
}

```

```

}

void handleMovement() {
    int xValue = data.x;
    int yValue = data.y;

    int xMapped = map(xValue, 0, 1023, -512, 512);
    int yMapped = map(yValue, 0, 1023, 512, -512); // inverted to fix
forward/backward

    int deadZone = 100;
    if (abs(xMapped) < deadZone) xMapped = 0;
    if (abs(yMapped) < deadZone) yMapped = 0;

    int leftSpeed = yMapped + xMapped;
    int rightSpeed = yMapped - xMapped;

    leftSpeed = constrain(leftSpeed, -512, 512);
    rightSpeed = constrain(rightSpeed, -512, 512);

    setMotor(ENA, IN1, IN2, leftSpeed);
    setMotor(ENB, IN3, IN4, rightSpeed);
}

void setMotor(int EN, int IN1, int IN2, int speedVal) {
    if (speedVal > 0) {
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
    } else if (speedVal < 0) {
        digitalWrite(IN1, LOW);
        digitalWrite(IN2, HIGH);
    } else {
        digitalWrite(IN1, LOW);
        digitalWrite(IN2, LOW);
    }
    analogWrite(EN, map(abs(speedVal), 0, 512, 0, 255));
}

void handleServo() {
    if (data.button && !prevButton) {
        myServo.write(90);
        delay(250);
        myServo.write(0);
    }
    prevButton = data.button;
}

```

```
}
```

Controller Code

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

RF24 radio(9, 10); // CE, CSN
const byte address[6] = "00001";

// Pins
#define JOY_X A0
#define JOY_Y A1
#define KICKER_BTN 4
#define GOALIE_X A2
#define GOALIE_BTN 5

struct DataPacket {
    int x;
    int y;
    bool button;
    int x2;
    bool goalieBtn;
};

DataPacket data;

void setup() {
    pinMode(KICKER_BTN, INPUT_PULLUP);
    pinMode(GOALIE_BTN, INPUT_PULLUP);

    radio.begin();
    radio.openWritingPipe(address);
    radio.setPALevel(RF24_PA_LOW);
    radio.stopListening();
}

void loop() {
    data.x = analogRead(JOY_X);
    data.y = analogRead(JOY_Y);
    data.button = (digitalRead(KICKER_BTN) == LOW);
    data.x2 = analogRead(GOALIE_X);
    data.goalieBtn = (digitalRead(GOALIE_BTN) == LOW);
```

```
radio.write(&data, sizeof(data));
delay(20);
}
```

Arena Code

```
/*
 * Football Arena Controller – FINAL VERSION
 * Hardware: ESP32 + 2x IR Break Beam Sensors + 2x LED Strips (90 bulbs each) +
Active Piezo Buzzer
 *
 * PIN LAYOUT:
 *   IR Sensor 1 (Player 1 goal)  -> GPIO 26
 *   IR Sensor 2 (Player 2 goal)  -> GPIO 27
 *   LED Strip 1 (Player 1 side)   -> GPIO 5
 *   LED Strip 2 (Player 2 side)   -> GPIO 18
 *   Piezo Buzzer                  -> GPIO 19
 *
 * WIRING:
 *   IR Sensors : VCC -> 3.3V, GND -> GND, OUT -> GPIO
 *   LED Strips : External 5V PSU -> Strip VCC
 *               External GND + ESP32 GND joined together -> Strip GND
 *               470Ω resistor on data line (GPIO -> Strip DIN)
 *   Buzzer      : GPIO 19 -> positive leg, GND -> negative leg
 *               HIGH = ON (active buzzer)
 */

#include <WiFi.h>
#include <WebServer.h>
#include <FastLED.h>
#include <ArduinoJson.h>

// ---- WiFi Credentials ----
const char* ssid      = "REDMI 15C";
const char* password  = "123456789";

// ---- Pin Definitions ----
#define IR_SENSOR_1    26
#define IR_SENSOR_2    27
#define LED_STRIP_1     5
#define LED_STRIP_2    18
#define BUZZER_PIN     19
```

```

// ---- LED Config ----
#define NUM_LEDS      90
#define LED_TYPE      WS2812B
#define COLOR_ORDER   GRB
#define LED_BRIGHTNESS 80      // Keep low until external PSU is wired
properly

CRGB strip1[NUM_LEDS];
CRGB strip2[NUM_LEDS];

// ---- Web Server ----
WebServer server(80);

// ---- Game State ----
struct GameState {
    bool      active      = false;
    bool      paused      = false;
    int       timeLeft    = 0;
    int       score1      = 0;
    int       score2      = 0;
    String    name1       = "Player 1";
    String    name2       = "Player 2";
    int       totalTime   = 0;
    unsigned long lastTick = 0;
    unsigned long pauseEnd = 0;
    int       goalWinner  = 0;
};

GameState game;

// ---- IR Edge Detection ----
bool sensor1Clear = true;
bool sensor2Clear = true;

// ---- LED task state ----
// We drive LEDs from a separate FreeRTOS task to avoid WiFi interference
enum LedMode {
    LED_OFF,
    LED_RAINBOW,
    LED_WHITE,
    LED_GOAL_1,    // strip1 red, strip2 off
    LED_GOAL_2,    // strip2 red, strip1 off
    LED_WIN_1,     // strip1 green, strip2 red
    LED_WIN_2,     // strip1 red, strip2 green
    LED_DRAW       // both yellow
};

```

```

};

volatile LedMode ledMode = LED_RAINBOW; // rainbow on power-on
volatile uint8_t rainbowHue = 0;

// ---- Buzzer (non-blocking) ----
bool          buzzerActive = false;
unsigned long buzzerEnd    = 0;

// HIGH = ON for this hardware
void buzzerOn() { digitalWrite(BUZZER_PIN, HIGH); }
void buzzerOff() { digitalWrite(BUZZER_PIN, LOW); }

void startBuzzer(unsigned long ms) {
    buzzerOn();
    buzzerActive = true;
    buzzerEnd    = millis() + ms;
}

void updateBuzzer() {
    if (buzzerActive && millis() >= buzzerEnd) {
        buzzerOff();
        buzzerActive = false;
    }
}

// =====
// LED Task – runs on Core 0, WiFi runs on Core 1
// This prevents FastLED timing from being disrupted by WiFi
// =====
void ledTask(void* param) {
    FastLED.addLeds<LED_TYPE, LED_STRIP_1, COLOR_ORDER>(strip1,
NUM_LEDS).setCorrection(TypicalLEDStrip);
    FastLED.addLeds<LED_TYPE, LED_STRIP_2, COLOR_ORDER>(strip2,
NUM_LEDS).setCorrection(TypicalLEDStrip);
    FastLED.setBrightness(LED_BRIGHTNESS);
    fill_solid(strip1, NUM_LEDS, CRGB::Black);
    fill_solid(strip2, NUM_LEDS, CRGB::Black);
    FastLED.show();

    for (;;) {
        switch (ledMode) {
            case LED_RAINBOW:
                for (int i = 0; i < NUM_LEDS; i++) {
                    strip1[i] = CHSV(rainbowHue + (i * 256 / NUM_LEDS), 255, 200);

```

```

        strip2[i] = CHSV(rainbowHue + (i * 256 / NUM_LEDS), 255, 200);
    }
    FastLED.show();
    rainbowHue++;
    vTaskDelay(20 / portTICK_PERIOD_MS);
    break;

case LED_WHITE:
    fill_solid(strip1, NUM_LEDS, CRGB::White);
    fill_solid(strip2, NUM_LEDS, CRGB::White);
    FastLED.show();
    vTaskDelay(50 / portTICK_PERIOD_MS);
    break;

case LED_GOAL_1:
    fill_solid(strip1, NUM_LEDS, CRGB::Red);
    fill_solid(strip2, NUM_LEDS, CRGB::Black);
    FastLED.show();
    vTaskDelay(50 / portTICK_PERIOD_MS);
    break;

case LED_GOAL_2:
    fill_solid(strip2, NUM_LEDS, CRGB::Red);
    fill_solid(strip1, NUM_LEDS, CRGB::Black);
    FastLED.show();
    vTaskDelay(50 / portTICK_PERIOD_MS);
    break;

case LED_WIN_1:
    fill_solid(strip1, NUM_LEDS, CRGB::Green);
    fill_solid(strip2, NUM_LEDS, CRGB::Red);
    FastLED.show();
    vTaskDelay(50 / portTICK_PERIOD_MS);
    break;

case LED_WIN_2:
    fill_solid(strip1, NUM_LEDS, CRGB::Red);
    fill_solid(strip2, NUM_LEDS, CRGB::Green);
    FastLED.show();
    vTaskDelay(50 / portTICK_PERIOD_MS);
    break;

case LED_DRAW:
    fill_solid(strip1, NUM_LEDS, CRGB::Yellow);
    fill_solid(strip2, NUM_LEDS, CRGB::Yellow);

```



```

        FastLED.show();
        vTaskDelay(50 / portTICK_PERIOD_MS);
        break;

    case LED_OFF:
    default:
        fill_solid(strip1, NUM_LEDS, CRGB::Black);
        fill_solid(strip2, NUM_LEDS, CRGB::Black);
        FastLED.show();
        vTaskDelay(100 / portTICK_PERIOD_MS);
        break;
    }
}
}

// =====
//  CORS
//  =====
void addCORS() {
    server.setHeader("Access-Control-Allow-Origin", "*");
    server.setHeader("Access-Control-Allow-Methods", "GET, POST, OPTIONS");
    server.setHeader("Access-Control-Allow-Headers", "Content-Type");
}

// =====
//  API: POST /api/start
//  =====
void handleStart() {
    addCORS();
    if (server.method() == HTTP_OPTIONS) { server.send(204); return; }
    if (!server.hasArg("plain")) {
        server.send(400, "application/json", "{\"error\":\"no body\"}");
        return;
    }

    StaticJsonDocument<256> doc;
    if (deserializeJson(doc, server.arg("plain"))) {
        server.send(400, "application/json", "{\"error\":\"bad json\"}");
        return;
    }

    game.name1      = doc["name1"] | "Player 1";
    game.name2      = doc["name2"] | "Player 2";
    game.totalTime  = doc["time"]  | 60;
    game.timeLeft   = game.totalTime;

```

```

game.score1    = 0;
game.score2    = 0;
game.active    = true;
game.paused    = false;
game.lastTick  = millis();
game.goalWinner = 0;

sensor1Clear = true;
sensor2Clear = true;

// Both strips white when game starts
ledMode = LED_WHITE;

// Beep to signal game start
startBuzzer(500);

server.send(200, "application/json", "{\"status\":\"started\"}");
Serial.println("Game started!");
}

// =====
//  API: POST /api/stop
//  =====
void handleStop() {
    addCORS();
    game.active = false;
    game.paused = false;
    buzzerOff();
    buzzerActive = false;
    ledMode      = LED_RAINBOW; // back to rainbow when stopped
    server.send(200, "application/json", "{\"status\":\"stopped\"}");
}

// =====
//  API: GET /api/state
//  =====
void handleState() {
    addCORS();
    StaticJsonDocument<256> doc;
    doc["active"]    = game.active;
    doc["paused"]    = game.paused;
    doc["timeLeft"]  = game.timeLeft;
    doc["score1"]    = game.score1;
    doc["score2"]    = game.score2;
    doc["name1"]     = game.name1;

```

```

    doc["name2"]    = game.name2;
    doc["total"]    = game.totalTime;
    doc["winner"]   = game.goalWinner;
    String out;
    serializeJson(doc, out);
    server.send(200, "application/json", out);
}

void handleOptions() {
    addCORS();
    server.send(204);
}

// =====
// Goal trigger
// =====
void triggerGoal(int player) {
    if (game.paused || !game.active) return;

    if (player == 1) {
        game.score1++;
        sensor1Clear = false;
        ledMode       = LED_GOAL_1;    // strip1 red, strip2 off
    } else {
        game.score2++;
        sensor2Clear = false;
        ledMode       = LED_GOAL_2;    // strip2 red, strip1 off
    }

    startBuzzer(3000);                // buzzer on for 3 seconds
    game.paused    = true;
    game.goalWinner = player;
    game.pauseEnd   = millis() + 3000;

    Serial.printf("GOAL! Player %d | Score: %d - %d\n", player, game.score1,
game.score2);
}

// =====
// Resume after 3s pause
// =====
void resumeGame() {
    game.paused    = false;
    game.goalWinner = 0;
    sensor1Clear   = true;

```

```

    sensor2Clear    = true;
    ledMode         = LED_WHITE;    // both strips white
    startBuzzer(200);                // short beep on resume
    Serial.println("Game resumed.");
}

// =====
//  Game over
//  =====
void endGame() {
    game.active = false;
    game.paused = false;

    if (game.score1 > game.score2)    ledMode = LED_WIN_1;
    else if (game.score2 > game.score1) ledMode = LED_WIN_2;
    else                             ledMode = LED_DRAW;

    // 3 victory beeps
    for (int i = 0; i < 3; i++) {
        buzzerOn(); delay(300);
        buzzerOff(); delay(200);
    }

    Serial.printf("Game Over!  %s %d - %d %s\n",
        game.name1.c_str(), game.score1, game.score2, game.name2.c_str());
}

// =====
//  SETUP
//  =====
void setup() {
    Serial.begin(115200);

    pinMode(IR_SENSOR_1, INPUT_PULLUP);
    pinMode(IR_SENSOR_2, INPUT_PULLUP);
    pinMode(BUZZER_PIN,  OUTPUT);
    buzzerOff();

    // Start LED task on Core 0 (WiFi uses Core 1)
    xTaskCreatePinnedToCore(
        ledTask,        // function
        "LEDTask",      // name
        4096,           // stack size
        NULL,           // params
        1,              // priority

```

```

    NULL,          // handle
    0              // core 0
);

// Brief buzzer beep on boot
buzzerOn(); delay(200); buzzerOff();

// WiFi
Serial.printf("Connecting to %s", ssid);
WiFi.disconnect(true);
delay(500);
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);

int attempts = 0;
while (WiFi.status() != WL_CONNECTED && attempts < 20) {
    delay(500);
    Serial.print(".");
    attempts++;
}

if (WiFi.status() == WL_CONNECTED) {
    Serial.printf("\nConnected! IP: %s\n", WiFi.localIP().toString().c_str());
} else {
    Serial.println("\nWiFi failed - check SSID/password.");
}

server.on("/api/start", HTTP_POST,    handleStart);
server.on("/api/start", HTTP_OPTIONS, handleOptions);
server.on("/api/stop",  HTTP_POST,    handleStop);
server.on("/api/stop",  HTTP_OPTIONS, handleOptions);
server.on("/api/state", HTTP_GET,     handleState);
server.on("/api/state", HTTP_OPTIONS, handleOptions);
server.begin();

Serial.println("Server ready.");
Serial.println(WiFi.localIP());
}

// =====
// LOOP - game logic only, LEDs handled by ledTask
// =====
void loop() {
    server.handleClient();
    updateBuzzer();

```

```

if (!game.active) return;

unsigned long now = millis();

// Goal celebration pause
if (game.paused) {
    if (now >= game.pauseEnd) resumeGame();
    return;
}

// Countdown tick
if (now - game.lastTick >= 1000) {
    game.lastTick = now;
    if (game.timeLeft > 0) {
        game.timeLeft--;
        if (game.timeLeft == 0) {
            endGame();
            return;
        }
    }
}

// IR sensor polling with edge detection
bool s1 = (digitalRead(IR_SENSOR_1) == LOW);
bool s2 = (digitalRead(IR_SENSOR_2) == LOW);

if (!s1) sensor1Clear = true;
if (!s2) sensor2Clear = true;

if (s1 && sensor1Clear) triggerGoal(1);
if (s2 && sensor2Clear) triggerGoal(2);
}

```

Website code

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Football Arena</title>

```

```

<link
href="https://fonts.googleapis.com/css2?family=Bebas+Neue&family=Inter:wght@400;500;600&display=swap" rel="stylesheet">
<style>
  *{box-sizing:border-box;margin:0;padding:0}
  :root{--g:linear-gradient(135deg,#0f0f0f 0%,#1a1a2e 50%,#16213e 100%);--
accent:#00ff88;--red:#ff3b5c;--gold:#ffd700;--card:rgba(255,255,255,0.04);--
border:rgba(255,255,255,0.08)}
  body{background:var(--g);min-height:100vh;font-family:'Inter',sans-
serif;color:#fff;display:flex;flex-direction:column;align-
items:center;padding:2rem 1rem}
  h1{font-family:'Bebas Neue',sans-serif;font-size:3.2rem;letter-
spacing:4px;background:linear-gradient(90deg,#00ff88,#00ccff);-webkit-background-
clip:text;-webkit-text-fill-color:transparent;margin-bottom:.25rem;text-
align:center}
  .subtitle{font-size:.75rem;letter-spacing:3px;color:rgba(255,255,255,.35);text-
transform:uppercase;margin-bottom:2rem;text-align:center}
  #setup{width:100%;max-width:520px;display:flex;flex-
direction:column;gap:1.25rem}
  .field-group{display:flex;flex-direction:column;gap:.4rem}
  label{font-size:.7rem;letter-spacing:2px;text-
transform:uppercase;color:rgba(255,255,255,.45)}
  input[type=text],input[type=number]{background:var(--card);border:1px solid
var(--border);border-radius:8px;padding:.75rem 1rem;color:#fff;font-
size:.95rem;font-family:inherit;outline:none;transition:border-color
.2s;width:100%}
  input:focus{border-color:var(--accent)}
  .row{display:grid;grid-template-columns:1fr 1fr;gap:1rem}
  .player-card{background:var(--card);border:1px solid var(--border);border-
radius:12px;padding:1.25rem;display:flex;flex-direction:column;gap:.75rem}
  .dot{width:10px;height:10px;border-radius:50%;margin-bottom:.25rem}
  .dot-green{background:var(--accent);box-shadow:0 0 8px var(--accent)}
  .dot-red{background:var(--red);box-shadow:0 0 8px var(--red)}
  .btn-start{background:linear-gradient(135deg,var(--
accent),#00ccff);color:#000;border:none;border-radius:10px;padding:1rem;font-
size:1rem;font-weight:700;letter-spacing:2px;text-
transform:uppercase;cursor:pointer;transition:transform .15s,opacity .2s;margin-
top:.5rem}
  .btn-start:hover{transform:scale(1.02)}
  .btn-start:active{transform:scale(0.98)}
  .btn-start:disabled{opacity:.4;cursor:not-allowed;transform:none}
  .ip-row{display:flex;gap:.75rem;align-items:flex-end}
  .ip-row input{flex:1}

```

```

    .btn-connect{background:var(--card);border:1px solid var(--border);border-
radius:8px;padding:.75rem 1.25rem;color:#fff;font-size:.8rem;font-
family:inherit;cursor:pointer;white-space:nowrap;transition:border-color .2s}
    .btn-connect:hover{border-color:var(--accent)}
    .status-dot{width:8px;height:8px;border-
radius:50%;background:#555;display:inline-block;margin-right:6px;vertical-
align:middle;transition:background .3s}
    .status-dot.ok{background:var(--accent);box-shadow:0 0 6px var(--accent)}
    .status-dot.err{background:var(--red);box-shadow:0 0 6px var(--red)}
    .status-label{font-size:.72rem;letter-
spacing:1px;color:rgba(255,255,255,.4);display:flex;align-items:center}
    #game{width:100%;max-width:700px;display:none;flex-direction:column;gap:1.5rem}
    .timer-ring{text-align:center}
    .timer-num{font-family:'Bebas Neue',sans-serif;font-size:5rem;letter-
spacing:4px;line-height:1}
    .timer-label{font-size:.65rem;letter-
spacing:3px;color:rgba(255,255,255,.35);text-transform:uppercase;margin-
top:.25rem}
    .scoreboard{display:grid;grid-template-columns:1fr auto 1fr;gap:.5rem;align-
items:center}
    .player-panel{background:var(--card);border:1px solid var(--border);border-
radius:16px;padding:1.5rem 1rem;display:flex;flex-direction:column;align-
items:center;gap:.5rem;transition:all .3s;min-height:160px;justify-
content:center}
    .player-panel.scored{border-color:var(--
red)!important;background:rgba(255,59,92,.08)!important}
    .player-panel.winner{border-color:var(--gold);background:rgba(255,215,0,.06)}
    .pname{font-size:.8rem;letter-spacing:2px;text-
transform:uppercase;color:rgba(255,255,255,.5)}
    .pscore{font-family:'Bebas Neue',sans-serif;font-size:4.5rem;line-
height:1;transition:transform .2s}
    .pscore.bump{transform:scale(1.25)}
    .pled{width:100%;height:6px;border-radius:3px;background:#333;overflow:hidden}
    .pled-fill{height:100%;border-radius:3px;transition:background .4s}
    .vs{font-family:'Bebas Neue',sans-serif;font-
size:1.8rem;color:rgba(255,255,255,.2);text-align:center}
    .led-strips{display:grid;grid-template-columns:1fr 1fr;gap:.75rem}
    .strip-wrap{border-radius:8px;overflow:hidden;height:12px;border:1px solid
var(--border)}
    .strip-inner{height:100%;width:100%;transition:background .4s}
    .strip-label{font-size:.62rem;letter-spacing:2px;text-
transform:uppercase;color:rgba(255,255,255,.3);margin-bottom:.35rem}
    .goal-banner{text-align:center;padding:.75rem;border-radius:10px;font-
family:'Bebas Neue',sans-serif;font-size:2rem;letter-

```



```

spacing:4px;background:rgba(255,59,92,.12);border:1px solid
rgba(255,59,92,.3);color:var(--red);display:none}
.goal-banner.show{display:block;animation:goalPulse 1s ease infinite}
@keyframes goalPulse{0%,100%{opacity:1}50%{opacity:.5}}
.gameover-panel{text-align:center;padding:2rem;background:var(--
card);border:1px solid var(--border);border-radius:16px;display:none;flex-
direction:column;gap:.75rem;align-items:center}
.gameover-panel.show{display:flex}
.go-title{font-family:'Bebas Neue',sans-serif;font-size:3rem;letter-
spacing:4px;color:var(--gold)}
.go-winner{font-size:1.1rem;font-weight:600}
.btn-reset{background:var(--card);border:1px solid var(--border);border-
radius:8px;padding:.75rem 2rem;color:#fff;font-size:.85rem;letter-
spacing:2px;text-transform:uppercase;cursor:pointer;font-
family:inherit;transition:border-color .2s;margin-top:.5rem}
.btn-reset:hover{border-color:var(--accent)}
.err-msg{color:var(--red);font-size:.75rem;letter-spacing:1px;text-
align:center;min-height:1rem}
</style>
</head>
<body>
<h1>Football Arena</h1>
<p class="subtitle">Powered by ESP32</p>
<div id="setup">
  <div class="field-group">
    <label>ESP32 IP Address</label>
    <div class="ip-row">
      <input type="text" id="ipInput" placeholder="192.168.1.100">
      <button class="btn-connect" onclick="testConnection()">Connect</button>
    </div>
    <div class="status-label"><span class="status-dot" id="connDot"></span><span
id="connText">Not connected</span></div>
  </div>
  <div class="row">
    <div class="player-card"><div class="dot dot-green"></div><div class="field-
group"><label>Player 1 Name</label><input type="text" id="p1name"
placeholder="Alice" maxlength="16"></div></div>
    <div class="player-card"><div class="dot dot-red"></div><div class="field-
group"><label>Player 2 Name</label><input type="text" id="p2name"
placeholder="Bob" maxlength="16"></div></div>
  </div>
  <div class="field-group"><label>Game Duration (seconds)</label><input
type="number" id="gametime" value="120" min="10" max="600"></div>
  <p class="err-msg" id="errMsg"></p>

```

```

    <button class="btn-start" id="btnStart" onclick="startGame()">Start
Game</button>
</div>
<div id="game">
    <div class="timer-ring"><div class="timer-num" id="timerNum">0:00</div><div
class="timer-label">Time Remaining</div></div>
    <div class="goal-banner" id="goalBanner">GOAL!</div>
    <div class="scoreboard">
        <div class="player-panel" id="panel1"><div class="pname" id="gname1">Player
1</div><div class="pscore" id="gscore1">0</div><div class="pled"><div
class="pled-fill" id="led1bar" style="width:6px;background:var(--
accent)"></div></div></div>
        <div class="vs">VS</div>
        <div class="player-panel" id="panel2"><div class="pname" id="gname2">Player
2</div><div class="pscore" id="gscore2">0</div><div class="pled"><div
class="pled-fill" id="led2bar" style="width:6px;background:var(--
red)"></div></div></div>
    </div>
    <div>
        <div class="strip-label">LED Strips</div>
        <div class="led-strips">
            <div><div class="strip-label" id="slabel1">Player 1</div><div class="strip-
wrap"><div class="strip-inner" id="strip1vis"></div></div></div>
            <div><div class="strip-label" id="slabel2">Player 2</div><div class="strip-
wrap"><div class="strip-inner" id="strip2vis"></div></div></div>
        </div>
    </div>
    <div class="gameover-panel" id="gameoverPanel">
        <div class="go-title">Full Time!</div>
        <div class="go-winner" id="goWinner"></div>
        <button class="btn-reset" onclick="resetGame()">New Game</button>
    </div>
</div>
<script>
let
esp32IP='',pollTimer=null,lastScore1=0,lastScore2=0,inRainbow=false,rainbowInterv
al=null,gameRunning=false,rainbowHue=0;
function getBase(){return`http://${esp32IP}`}
async function testConnection(){
    const ip=document.getElementById('ipInput').value.trim();
    if(!ip){showErr('Enter IP address');return}
    esp32IP=ip;
    const
dot=document.getElementById('connDot'),txt=document.getElementById('connText');
    txt.textContent='Connecting...';dot.className='status-dot';

```

```

    try{const r=await
fetch(`http://${ip}/api/state`,{signal:AbortSignal.timeout(3000)});
    if(r.ok){dot.className='status-dot ok';txt.textContent=`Connected to
${ip}`}else throw new Error()
    }catch{dot.className='status-dot err';txt.textContent='Connection failed -
check IP';esp32IP=''}}
}
function
showErr(msg){document.getElementById('errMsg').textContent=msg;setTimeout(==>doc
ument.getElementById('errMsg').textContent='',3000)}
async function startGame(){
    const ip=document.getElementById('ipInput').value.trim();
    if(!ip){showErr('Connect to ESP32 first');return}
    esp32IP=ip;
    const n1=document.getElementById('p1name').value.trim()||'Player 1';
    const n2=document.getElementById('p2name').value.trim()||'Player 2';
    const t=parseInt(document.getElementById('gametime').value)||120;
    document.getElementById('btnStart').disabled=true;
    try{
        const r=await
fetch(`${getBase()}/api/start`,{method:'POST',headers:{'Content-
Type':'application/json'},body:JSON.stringify({name1:n1,name2:n2,time:t})});
        if(!r.ok)throw new Error();
        lastScore1=0;lastScore2=0;
        ['gname1','gname2'].forEach((id,i)=>document.getElementById(id).textContent=[
n1,n2][i]);
        ['slabel1','slabel2'].forEach((id,i)=>document.getElementById(id).textContent
=[n1,n2][i]);
        ['gscore1','gscore2'].forEach(id=>document.getElementById(id).textContent='0'
);
        document.getElementById('gameoverPanel').classList.remove('show');
        document.getElementById('goalBanner').classList.remove('show');
        document.getElementById('setup').style.display='none';
        document.getElementById('game').style.display='flex';
        gameRunning=true;
        startRainbow();
        setTimeout(==>{stopRainbow();setStripsWhite()},3000);
        startPolling();
    }catch{showErr('Failed to start - check
connection');document.getElementById('btnStart').disabled=false}
}
function
startPolling(){if(pollTimer)clearInterval(pollTimer);pollTimer=setInterval(poll,5
00)}
async function poll(){

```

```

    if(!gameRunning)return;
    try{const r=await
fetch(`${getBase()}/api/state`,{signal:AbortSignal.timeout(2000)});updateUI(await
r.json())}catch{}
}
function updateUI(d){
    const m=Math.floor(d.timeLeft/60),s=d.timeLeft%60;
    document.getElementById('timerNum').textContent=`${m}:${s.toString().padStart(2
,'0')}`;
    if(d.score1!==lastScore1){document.getElementById('gscore1').textContent=d.scor
e1;bumpAnim('gscore1');lastScore1=d.score1}
    if(d.score2!==lastScore2){document.getElementById('gscore2').textContent=d.scor
e2;bumpAnim('gscore2');lastScore2=d.score2}
    const total=Math.max(d.score1+d.score2,1);
    document.getElementById('led1bar').style.width=`${Math.max(8,(d.score1/total)*1
00)}%`;
    document.getElementById('led2bar').style.width=`${Math.max(8,(d.score2/total)*1
00)}%`;
    const
p1=document.getElementById('panel1'),p2=document.getElementById('panel2'),banner=
document.getElementById('goalBanner');
    if(d.paused&&d.winner>0){
        banner.textContent=`GOAL!  ${d.winner==1?d.name1:d.name2} scored!`;
        banner.classList.add('show');
        if(d.winner==1){p1.classList.add('scored');p2.classList.remove('scored');setS
trip1('red');setStrip2('off')}
        else{p2.classList.add('scored');p1.classList.remove('scored');setStrip2('red'
);setStrip1('off')}
    }else if(!d.paused&&d.active){
        banner.classList.remove('show');p1.classList.remove('scored');p2.classList.re
move('scored');
        if(!inRainbow){setStrip1('white');setStrip2('white')}
    }
    if(!d.active&&gameRunning){
        gameRunning=false;stopRainbow();clearInterval(pollTimer);
        banner.classList.remove('show');p1.classList.remove('scored');p2.classList.re
move('scored');
        const
go=document.getElementById('gameoverPanel'),gw=document.getElementById('goWinner'
);
        if(d.score1>d.score2){gw.textContent=`${d.name1} wins!  ${d.score1} -
${d.score2}`;setStrip1('green');setStrip2('red');p1.classList.add('winner')}
        else if(d.score2>d.score1){gw.textContent=`${d.name2} wins!  ${d.score2} -
${d.score1}`;setStrip2('green');setStrip1('red');p2.classList.add('winner')}

```

```

        else{gw.textContent=`It's a draw! ${d.score1} -
        ${d.score2}`;setStrip1('yellow');setStrip2('yellow')}
        go.classList.add('show');
    }
}
function bumpAnim(id){const
el=document.getElementById(id);el.classList.add('bump');setTimeout(()=>el.classLi
st.remove('bump'),300)}
function startRainbow(){
    inRainbow=true;if(rainbowInterval)clearInterval(rainbowInterval);
    rainbowInterval=setInterval(()=>{
        rainbowHue=(rainbowHue+3)%360;
        document.getElementById('strip1vis').style.background=`hsl(${rainbowHue},100%
,50%)`;
        document.getElementById('strip2vis').style.background=`hsl(${(rainbowHue+120)
%360},100%,50%)`;
    },40);
}
function
stopRainbow(){inRainbow=false;if(rainbowInterval){clearInterval(rainbowInterval);
rainbowInterval=null}}
function
setStripsWhite(){document.getElementById('strip1vis').style.background='#ffffff';
document.getElementById('strip2vis').style.background='#ffffff'}
function
setStrip1(c){document.getElementById('strip1vis').style.background=colorMap(c)}
function
setStrip2(c){document.getElementById('strip2vis').style.background=colorMap(c)}
function
colorMap(c){return{white:'#ffffff',red:'#ff3b5c',off:'#111111',green:'#00ff88',ye
llow:'#ffd700'}[c]||'#111'}
async function resetGame(){
    try{await fetch(`${getBase()}/api/stop`,{method:'POST'})}catch{}
    gameRunning=false;stopRainbow();if(pollTimer)clearInterval(pollTimer);
    ['panel1','panel2'].forEach(id=>document.getElementById(id).classList.remove('w
inner','scored'));
    document.getElementById('strip1vis').style.background='#111';
    document.getElementById('strip2vis').style.background='#111';
    document.getElementById('game').style.display='none';
    document.getElementById('setup').style.display='flex';
    document.getElementById('btnStart').disabled=false;
}
</script>
</body>
</html>

```

